

Network Topology Aware Negative Sampling for Improving Recommendation

The effectiveness of collaborative filtering methods depends not only on the expressive power of interaction data but also on the model’s ability to learn meaningful representations. Among the factors influencing this process, the strategy used for negative sampling plays a crucial role in guiding representation learning. Despite the inherently complex and structured nature of user-item interactions, existing negative sampling approaches fail to exploit the topological characteristics of such systems. In this study, we propose ten novel network-aware negative sampling strategies based on key topological concepts from complex networks, specifically path length and path density. Our experimental evaluation on the LightGCN model using the LastFM dataset shows that incorporating these topology-informed techniques yields substantial improvements—up to 6.7% gain in Precision@10, Recall@10, and NDCG@10—compared to standard random negative sampling. Additionally, our methods significantly reduce training time by up to 30–35%. These findings demonstrate the untapped potential of leveraging complex network topology to enhance learning processes in state-of-the-art recommender systems.

1. Introduction

1.1. Overview

Recommendation systems (RS) play a crucial role in assisting users with decision-making across a variety of applications, from e-commerce to entertainment services.¹ Their performance, however, lies in the quality and quantity of the data available about the user and their interaction with the studied system. In recent years, collaborative filtering (CF) algorithms that naturally depend on user-item interactions, have gained popularity due to their significant performance in RS.² These approaches utilize explicit interactions, such as ratings or likes/dislikes, which reveal user preferences directly, or implicit interactions, like purchases or clicks, which indicate interest but not explicit preference.³

While recent machine learning techniques are effective in uncovering patterns among the features of individual objects, they often overlook the interactions between these objects. However, such interactions—and the dynamic processes they generate—play a crucial role in shaping decisions within RS. In this context, incorporating complex network-based concepts⁴ into recommendation strategies, particularly collaborative filtering approaches, can offer deeper insights and enhanced performance compared to using raw machine learning methods alone.

When interpreting user-item interactions as a complex system, two major challenges emerge that significantly affect the performance of any recommendation model: (i) data sparsity and (ii) class imbalance. *Data sparsity* arises when users interact with only a small subset of the available items, resulting in incomplete and sparse interaction data. This limits the system’s ability to generate accurate and per-

sonalized recommendations, often leading to irrelevant suggestions and a poor user experience. A frequent consequence of data sparsity, *class imbalance* occurs when the number of interacted items (class 1) is vastly outnumbered by non-interacted items (class 0). This imbalance hinders the algorithm’s capacity to accurately learn user preferences. This paper addresses these two challenges with the aim of enhancing the accuracy of RS by employing complex network-based techniques.

1.2. Problem Statement

One solution for the negative effects of data sparsity and class imbalance can be seen as content-based filtering (CBF) algorithms employing user attributes (age, gender, location) and item attributes (type, material, descriptions), enriching the context for understanding preferences and characteristics.⁵ This approach can be seen as employing purely machine learning techniques for prediction. However, they completely neglect user-item interactions and are too oriented to attribute similarity or patterns among commonly appearing attributes, which in turn leads them to recommend redundant items with no interest of users.⁶

One widely adopted approach to address these challenges is negative sampling,⁷ which involves selecting a subset of items that the user is unlikely to be interested in from the pool of non-interacted (negative) items. This technique helps CF algorithms learn user preferences more effectively. Recent models based on Graph Convolutional Networks (GCNs) commonly incorporate a negative sampling step during training as part of their architectural framework.^{2,8,9} The most prevalent strategy for negative sampling is uniform random sampling (RNS), where each non-interacted item has an equal probability of being selected as a negative sample. Despite its simplicity, RNS has notable limitations. It treats all non-interacted items as equally informative, regardless of whether the user is highly unlikely or potentially likely to interact with them. In other words, while the goal of negative sampling is to improve model learning, RNS does not take into account any contextual or system-dependent factors that could guide more meaningful sample selection. For example, system-specific characteristics—such as item popularity or user preferences—could influence the relevance of certain items as negative samples. In systems where such dynamics matter, RNS may lead to suboptimal learning by including too many irrelevant negatives.¹⁰

GCN-based collaborative filtering methods inherently leverage the bipartite structure of user-item interactions to model complex relationships. While GCN architectures utilize this network structure during the message-passing phase of learning, the bipartite network itself also serves as a powerful analytical representation of the entire system as a complex network. This perspective enables the extraction of rich topological features—such as centrality, connectivity, or structural roles—that may signal item relevance or influence. However, existing GCN-based recommender systems typically overlook this topological information when selecting negative samples. In this study, we aim to bridge this gap by integrating complex network con-

cepts—particularly path-based metrics such as path length and path density—into the negative sampling process to enhance the quality of training data in GCN-based recommendation models. Although leveraging network structure for designing effective negative sampling strategies has been previously proposed,¹¹ that work focuses on basic unsupervised node embedding methods such as Node2Vec,¹² LINE,¹³ and DeepWalk.¹⁴ However, these embedding techniques are not well-suited for personalized recommendation tasks due to their limited capacity to capture user-specific preferences.

We introduce novel negative sampling strategies that move beyond RNS by prioritizing structurally less relevant items based on their position within the user-item bipartite network specifically for personalized collaborative filtering. Our key insight is that the likelihood of a user interacting with an item can be inversely associated with specific topological indicators in the network. We define non-relevance using two primary metrics: (i) shortest path length, which reflects the minimum number of hops needed for a user to reach an item, and (ii) path density, which denotes the number of alternative paths connecting the user to the item. Intuitively, items that are farther away, i.e. longer path length, or reachable via fewer paths, i.e. lower path density, are less likely to be of interest. Based on these observations, we develop both deterministic and probabilistic sampling techniques. Deterministic strategies select, for instance, the farthest items, while probabilistic methods employ weighted sampling schemes where selection probabilities are proportional to these topological measures. This dual approach also allows us to systematically assess the impact of randomness in negative sampling, grounded in the structural properties of the underlying complex network.

1.3. Contributions

The primary contribution of this study lies in leveraging the topology of the user-item bipartite network to enhance the performance of GCN-based recommender systems. GCNs, as state-of-the-art deep learning architectures, exploit graph neighborhood structures to learn effective node embeddings and have shown remarkable success in recommendation tasks. Building upon this foundation, our work introduces a novel perspective by incorporating topological insights—such as path length and path density—into the negative sampling process. By selecting structurally more informative negative samples, we aim to guide the embedding learning phase more effectively. These topologically-aware sampling strategies enriches the training dynamics of GCNs and improves recommendation accuracy, marking a substantial step forward in integrating complex network principles into modern recommendation frameworks. While GCNs inherently utilize the network structure, they only focus on the first- and second-order neighborhoods of nodes during the learning process. In contrast, our approach incorporates a broader set of complex network-based topological features to enrich the representation of user-item relationships and ultimately improve recommendation performance. Besides, some other contributions

that this work proposes are as follows:

- We proposed 10 novel negative sampling techniques that utilize different aspects of network topology to mitigate the inefficiencies of RNS in sparse and class-imbalanced data environments.
- We integrate the proposed techniques into LightGCN, which is one of the best-performing GCN-based CF methods in the literature.⁸
- We evaluate the impact of proposed techniques on a well-known RS data set with the analysis of the accuracy and convergence time of LightGCN and the comparison with two baselines.
- We explore the effects of different levels of uniform selection in negative sampling by combining our proposed techniques with RNS.

In this regard, the subsequent sections of this paper are organized in the following order: Section 2 reviews related works, focusing on negative sampling techniques. Section 3 details our proposed method, introducing Path-Length Negative Sampling (PL-NS), Diffusion Distance Negative Sampling (DD-NS), and Path-Density Negative Sampling (PD-NS). Section 4 presents our experiments, including the experimental setup, dataset, baselines, and results, followed by an in-depth analysis. Finally, Section 5 concludes the paper with a summary of our findings and suggestions for future research directions.

2. Negative Sampling

2.1. Overview

We encounter various negative sampling techniques which are mainly used for facilitating artificial learning problems in different areas. Firstly, negative sampling emerged as a simplification of Noise Contrastive Estimation (NCE),¹⁵ converting probability estimation into a binary classification task. This technique gained prominence in models such as Word2Vec,¹⁶ where it significantly reduces the computational overhead of training by approximating the softmax function over large vocabularies. Instead of evaluating all possible words, negative sampling strategically focuses on a limited subset of positive context words and randomly chosen negative words, classifying them based on their likelihood of co-occurrence.¹⁵

The benefits of negative sampling include *(i) efficiency*, it minimizes training time and computational costs by reducing the number of negative examples, *(ii) class imbalance mitigation*, it reduces bias toward dominant classes by balancing the representation of positive and negative samples and *(iii) enhanced discrimination*, including samples similar to positive examples enhances the model's ability to differentiate between similar items.⁷

Fundamentally, negative sampling is important because the artificial learning model uses the selected samples to distinguish between positive pairs (relevant examples) and negative pairs (irrelevant examples). It models the whole studied system accordingly. Hence, it is critical to pick up the representative subset, especially

for the training phase. This process is guided by a loss function that maximizes the similarity of positive pairs while minimizing that of negative pairs. The commonly used loss functions are categorized into four groups: pair-based sampling, triple-based sampling, single positive with multiple negatives, and contrastive learning.¹⁰

For instance, Bayesian Personalized Ranking (BPR) loss from the first category prioritizes ranking positive samples higher than negative samples by maximizing their log-likelihood difference.¹⁷ Hinge loss from the second category focuses on triples, i.e., anchor, positive, and negative, by enforcing a margin-based constraint where the anchor’s similarity to the positive must exceed its similarity to the negative by a predefined margin.¹⁸ Cross-entropy loss from the third category identifies a single positive sample among a set of negatives, optimizing for the correct classification.¹⁸ Lastly, InfoNCE loss from the fourth category is specifically designed for contrastive frameworks, emphasizing the maximization of similarity within positive pairs and dissimilarity with negative pairs.¹⁹

In recommendation systems, negative sampling addresses both data sparsity and class imbalance. For each positive interaction (u, i) , where u and i is a user and a positive item respectively, a negative item j is sampled, and the pairwise ranking loss is computed with Eq. 1.

$$\text{Loss}_{uij} = -\log(\sigma(\hat{r}_{uij})) \quad (1)$$

$\hat{r}_{uij} = \hat{r}_{ui} - \hat{r}_{uj}$ is the difference in predicted scores for items i and j for user u . The sigmoid function $\sigma(x)$ ensures a probabilistic interpretation. The objective is to train the model such that the predicted score for positive interactions ($\hat{r}_{ui}$) consistently exceeds the score for negative interactions ($\hat{r}_{uj}$).

2.2. Evolution and Techniques of Negative Sampling

The process of designing an effective negative sampling strategy involves two critical decisions: (i) how to generate a candidate pool of negative examples, and (ii) how to select the most informative negatives from this pool. We summarize five key strategies for generating candidate negatives.¹⁰ **Global Selection** considers the entire dataset as the candidate pool for negative examples. While it provides a wide range of data points, it may include irrelevant negatives, leading to suboptimal learning. For instance, Word2vec¹⁵ uses the entire dictionary as the pool of negative samples, and LightGCN⁸ employs all unseen items as potential negatives. **Local Selection** narrows the candidate pool to specific subsets of the data, reducing computational overhead. For example, ANCE²⁰ selects the top-k negative examples based on the query, while DNS²¹ samples a random subset of negatives for computational efficiency. **Mini Batch Selection** uses pre-loaded data in the current batch, leveraging other samples within the batch as negatives. While computationally efficient, its effectiveness depends on batch size, with smaller batches potentially limiting negative diversity. **Hop Selection** is specifically designed for graph-structured data, this

method utilizes k -hop neighbors (nodes within k^{th} distance) as potential negative instances. However, for large datasets, this approach may become computationally infeasible due to its reliance on graph structure. **Memory-Based Selection** utilizes a memory bank to store negative examples from previous training iterations or batches. It offers a diverse pool of negatives and is particularly advantageous for contrastive learning tasks.

Static Negative Sampling (Static NS) Static NS methods, such as random negative sampling (RNS) and popularity-based negative sampling (PNS), are foundational approaches. RNS uniformly selects negative examples from the dataset, making it straightforward and computationally inexpensive. However, its random nature can include irrelevant negatives, limiting its learning potential. PNS improves upon RNS by biasing the selection toward popular items, which are less likely to be true positives. The probability of selecting an item v given user u is:

$$p(v|u) \propto \text{pop}(v)^a, \quad (2)$$

Hard Negative Sampling (Hard NS) Hard NS focuses on selecting challenging negatives that are close to the decision boundary, providing stronger learning signals. **Dynamic Negative Sampling (DNS)** DNS selects hard negatives based on the model's current state. It includes anchor-based DNS, which uses

feature space distances, and positive-based DNS, which focuses on negatives similar to positive examples. Anchor-positive DNS combines both approaches for improved performance.^{21, 23, 24}

- **Mixture-based DNS:** This technique synthesizes hard negatives in the embedding space by mixing existing negatives, often using methods inspired by Mixup.^{25, 26}

GAN-based Negative Sampling Generative Adversarial Networks (GANs) are used to generate hard negatives adversarially. A generator creates challenging negatives to "fool" a discriminator, which distinguishes between real positives and generated negatives. While effective, GAN-based NS increases computational complexity and training instability.^{27, 28}

Auxiliary-based Negative Sampling This approach leverages additional information, such as graph structures or metadata, to improve the quality of negative samples. Although it enhances performance, it requires more data and computational resources.¹⁰

In-batch Negative Sampling In-batch NS uses other samples within the same mini-batch as negatives, balancing efficiency and simplicity. Variants include:

- **Basic In-batch NS:** Treats all non-positive pairs in the batch as negatives, though it may introduce false negatives.⁹
- **Debiased In-batch NS:** Adjusts negative sample weights to mitigate false negatives.²⁹
- **Hard In-batch NS:** Focuses on hard negatives within the batch for more informative training.²⁰

3. Network-oriented Negative Sampling

In this section, we describe our proposed network-oriented negative sampling techniques. Unlike RNS or PNS, which treat data points independently, our proposed techniques leverage the underlying network structure. We treat the user-item interactions set as a bipartite network to reveal hidden connections among two different types of nodes, user-type and item-type. Our goal is to identify informative negative examples, i.e., non-interacted item-type nodes, by incorporating network properties like path lengths and densities to ensure more proper recommendations for the users. Like previous examples,^{8,9} we choose negative samples per user locally but take advantage of entire network topology globally at each selection. We propose three strategies; Path-Length Negative Sampling (PL-NS), Diffusion Distance-based Negative Sampling (DD-NS), and Path-Density Negative Sampling (PD-NS) and integrate them into the LightGCN model.

3.1. Path-Length Negative Sampling (PL-NS)

The PL-NS strategy leverages the shortest path, i.e., distance, within a network to identify informative negative examples for training an RS. Item nodes with concise paths are considered less informative negatives due to their potential similarity with the positive pair. Thus, this method prioritizes sampling negative item examples connected to the user node by longer distances, enhancing the model’s ability to distinguish between similar data points.

Firstly, we calculate the distance between each pair of nodes using Dijkstra’s algorithm. Then, for each target user, we randomly select a negative sample corresponding to each positive sample according to the length of the distance to the target user. Here, we propose two different ways. The first method, *long-distance weighted selection*, multiplies each item by its distance from the user, giving higher weight to items with longer distances. The second method, *quartile-based selection*, involves sorting the item distances. The focus is on Quartile 1 (Q1), representing the median of the first half of the sorted distances, and Quartile 2 (Q2), representing the overall median distances. These specific points in the sorted list correspond to particular path lengths away from the user. Items at these chosen distances are then randomly selected and assigned as negative items.

In summary, as seen in Figure 1, this technique prioritizes selecting negative items that are farther from the user in the network. While quartile-based selection targets specific distance ranges, long-distance weighted selection favors distant items through weighting. Both strategies aim to enhance the model’s ability to distinguish between positive and negative data points by using path length information in the negative sampling process.

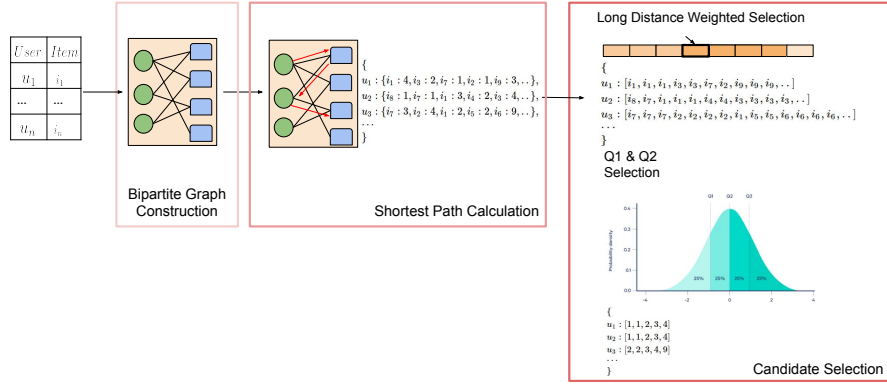


Fig. 1. Path-Length Negative Sampling Flow Chart

3.2. Diffusion Distance Negative Sampling (DD-NS)

The DD-NS approach measures node similarity by utilizing information propagation within a network through random walks. A random walk is a stochastic process where a walker moves randomly across the network, assessing how easily information spreads. In an undirected network $G = (V, E)$, each node $v \in V$ represents a position in the network, and each edge $(u, v) \in E$ represents a connection between nodes u and v . Starting at an initial node (v_0), the random walk moves to a neighbor with equal probability³⁰ (see Eq 3). The walk continues until a stopping requirement is met. Analyzing random walks provides insights into network activity, organizational structure, and vulnerability.³¹

$$P(v_i = v_j | v_{i-1} = v) = \begin{cases} \frac{1}{|N(v)|}, & \text{if } v_j \in N(v) \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

Here, $N(v)$ is the neighbor set of node v . To quantify node similarity, we employ a unique kind of random walk distance called the *diffusion distance*. We find more instructive negative examples using diffusion distance by taking into account both physical links and information flow. The procedure includes several key steps. Firstly, we calculate the adjacency matrix for network G with nodes $V = \{v_1, \dots, v_N\}$. The adjacency matrix $\mathbf{A}$ is $N \times N$ with $A_{ij} = 1$ if $\{i, j\} \in E$,

and 0 otherwise. Next, we compute the node degree of each node, where the degree k_i is the total number of neighbors of the node i , calculated as $k_i = \sum_{j=1}^N A_{ij}$. These degrees are collected into a diagonal matrix $\mathbf{D}$. Following this, we calculate the transition matrix as $\mathbf{T} = \mathbf{D}^{-1}\mathbf{A}$, where $T_{ij} = \frac{A_{ij}}{k_i}$ represents the probability of moving from node i to node j . The continuous-time Markov chain is then defined by the forward equation governing the continuous-time random walk:

$$\frac{d}{d\tau}\mathbf{p}(\tau) = -\mathbf{p}(\tau)\tilde{\mathbf{L}} \quad (4)$$

where $\tilde{\mathbf{L}} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$ is the normalized Laplacian of G . Given the initial condition $\mathbf{p}(\tau = 0) = \mathbf{e}_i$, the solution at time $\tau > 0$ is $p_{ij} = \left(e^{-\tau\tilde{\mathbf{L}}}\right)_{ij}$, representing the probability of the walker being at node j at time τ given it started at node i . The diffusion distance measures the distance between the posterior probability distributions of the walker at nodes i and j at time τ , calculated as:

$$D_\tau(i, j) = \|\mathbf{p}(\tau|i) - \mathbf{p}(\tau|j)\|_2 \quad (5)$$

This function finally calculates how different starting locations affect the walker's likelihood of being at various places after time τ . By analyzing the time parameter τ , we examine how information propagates. For each user, we select one negative item for each positive item considering diffusion distance, following five strategies as illustrated in Figure 2 (The images in the diffusion distance calculation process are courtesy from ^a). The first and second strategies are selecting the most distant

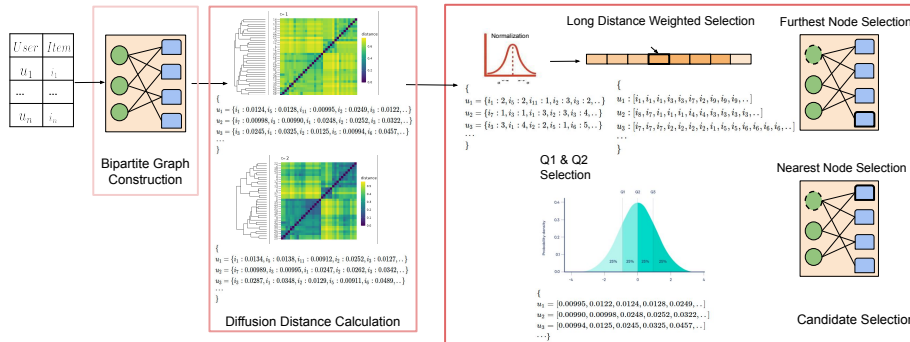


Fig. 2. Diffusion Distance Negative Sampling Flow Chart

(Farthest Node Selection) and the closest nodes (Nearest Node Selection) from the target user node as negative examples of item nodes respectively. The rest three

^a<https://gbertagnolli.github.io/diffust/d/articles/diffusion-distances.html#references-1>

strategies employ our proposed Long-Distance Weighted Selection and Quartile-Based Selection strategies as explained in Section 3.1. Unlike in Section 3.1, for Long-Distance Weighted Selection, we discretize the diffusion distances into different ranges since diffusion distance outputs values in the range $[0, 1]$. This operation allows us to prioritize items with larger diffusion distances by assigning them higher weights during random selection. Briefly, DD-NS strategies may be affected by two parameters; τ and scaling interval.

3.3. Path-Density Negative Sampling (PD-NS)

This approach incorporates path density between users and items. Path density refers to the relative number of paths connecting a user to an item within the bipartite network. The exact number of paths between two nodes in a network is computationally expensive ($\#P$ -complete).³² To address this, we estimate path density using random walks. This approach starts with a user and simulates a random walk on the network for a predetermined distance (`max_walk_length`). If the walk reaches a specific item, it is considered a successful path. We perform multiple random walks per user (`num_walks`) to get a more robust estimate. The pseudo-code of the path density calculation function is in the Algorithm 1.

Our approach utilizes two candidate selection methods: *max path density selection* and *min path density selection*. The max path density strategy prioritizes items with high path density, while the min path density strategy favors items with low path density. After removing any items the user has already interacted with, a random negative item is chosen from the remaining options.

4. Experiments and Results

In the experimental evaluation, we aim to discover the strengths and the limitations of the proposed negative sampling methods' around three research questions (RQs).

- **RQ1.** To what extent do our proposed network-oriented negative sampling techniques improve recommendation performance compared to the baselines, RNS, and PNS?
- **RQ2.** What is the effect of our proposed negative sampling approaches on the model's convergence?
- **RQ3.** How does varying the degree of randomness in our proposed negative sampling techniques influence recommendation performance?

To answer those questions we conduct a comprehensive set of experiments with a well-known dataset and evaluation metrics.

Dataset The experiments are performed on a well-known music recommendation dataset, Last.FM ^b. This dataset contains a collection of user-artist interactions,

^b<http://www.lastfm.com>

Algorithm 1 Estimate Paths

Require: Network G , source node $source$, target node $target$, number of walks num_walks , maximum walk length max_walk_length

Ensure: Estimated number of paths from $source$ to $target$ in G

```

1:  $num\_paths \leftarrow 0$ 
2: for  $walk \leftarrow 1$  to  $num\_walks$  do
3:    $current\_node \leftarrow source$ 
4:   for  $step \leftarrow 1$  to  $max\_walk\_length$  do
5:     if  $current\_node = target$  then
6:        $num\_paths \leftarrow num\_paths + 1$ 
7:       break
8:     end if
9:      $neighbors \leftarrow$  neighbors of  $current\_node$  in  $G$ 
10:    if  $neighbors$  is empty then
11:      break
12:    end if
13:     $current\_node \leftarrow$  randomly select a neighbor from  $neighbors$ 
14:  end for
15: end for
16: return  $num\_paths$ 

```

capturing listening behavior on the Last.fm music platform. It includes 1,892 users, 17,632 artists, and 92,834 user-artist interactions, encompassing a rich set of listening preferences. Its sparsity (99.72%) and class imbalance, make it a valuable resource for our research on RS.

Baseline We compare our proposed negative sampling approaches against two common negative sampling techniques: RNS and PNS. These baselines were chosen due to their widespread use and simplicity. RNS is the default negative sampler of LightGCN. The default version of both baselines picks only one positive and one negative per user for model learning.

Setup We integrated our proposed negative sampling approaches into the LightGCN⁸ framework to test their efficacy. We strictly obey the LightGCN’s original implementation using an embedding dimension of 64 with Xavier initialization for the embedding parameters,³³ and Adam³⁴ optimizer with a default learning rate of 0.001 and a mini-batch size of 2048. The L2 regularization coefficient λ was set to 0.0001, and the number of layers (K) remained at the default value of 3. Our code is publicly available at https://github.com/ElifEceErdem/network_oriented_NS. The experiments were conducted on a system with an Intel Xeon CPU E5-2697A v4 @ 2.60GHz and an NVIDIA Tesla P100-PCIE-12GB GPU. In order to have computational acceleration, we limited positive training examples to 10 per user, randomly

selecting if a user interacted with more than 10 items. In the experiments, we used the values $\{0.5, 1.0, 2.0, 4.0\}$ for τ of DD-NS. Besides, for the DD-NS-Long distance weighted strategy, three different scaling intervals, $[1, 5]$, $[1, 10]$, and $[1, 100]$, are employed to observe the sensitivity of the distance weighting. When calculating the path density for PD-NS, we used `num_walks=100` and `max_walk_length = 10` at the random walk calculation step. The performance of all experiments is evaluated using commonly used Precision@10, Recall@10, and NDCG@10.

4.1. The Performance of Proposed Methods

In order to answer the RQ1, we compare the proposed negative sampling approaches with baselines. As seen in table 1, the results for PD-based strategies underperform baseline methods. PD-Min has a similar Precision@10 to RNS (0.0804 vs. 0.0811) but lower Recall@10 and NDCG@10, indicating it misses some relevant negatives. PD-Max shows a significant drop in Precision@10 (0.0667), suggesting these are not effective negatives. Overall, path density is not a strong indicator for selecting effective negative items in this dataset.

Table 1. Proposed NS Approach Results with Baselines in Last.fm Dataset

Strategy	Sampling Approach	Precision@10	Recall@10	NDCG@10
Baseline	RNS	0.0811	0.1483	0.1347
	PNS	0.0790	0.1435	0.1295
Path-Length	PL-Long-Distance	0.0825	0.1496	0.1351
	PL-Q1	0.0797	0.1450	0.1324
	PL-Q2	0.0771	0.1379	0.1250
Diffusion Distance	DD-NS-Farthest	0.0109	0.0234	0.0175
	DD-NS-Nearest	0.0161	0.0318	0.0255
	DD-NS-Q1	0.0379	0.0673	0.0552
	DD-NS-Q2	0.0375	0.0674	0.0560
	DD-NS-Long-Distance	0.0830	0.1515	0.1382
Path-Density	PD-Max	0.0667	0.1237	0.0186
	PD-Min	0.0804	0.1437	0.1314

Among the other two strategies, PL-based strategies indicate that the Long-Distance method outperforms RNS and PNS, suggesting that items positioned further away from the user in the network are more informative negatives. However, quartile-based strategies (Q1 and Q2), focusing on specific distances, underperformed likely because they exclude potentially informative negatives at other path lengths. Specifically, items that are further away might better represent the concept of negative examples by being less similar to the user’s interests.

For the DD-NS approach, notably, using the nearest node significantly hurts

the model’s performance. They are the items that are more likely to be of interest to the user. Obviously, they are ineffective negative examples. Both the nearest and farthest node strategies perform worse than the baselines. Nevertheless, we observed a similar success of DD-NS-Long-Distance with PL-Long-Distance. The DD-NS-Long-Distance scaled at $[1, 5]$ outperformed the baselines and the other strategies across all evaluation metrics. Although prioritizing the farthest items in PL-Long-Distance and DD-NS-Long-Distance seems promising, the results of DD-NS-Farthest suggest that far-away items may lack context or relevance. In fact, we still use weighted uniform random selection in the long-distance strategies, while only the most distant items are selected in the farthest strategy. This may indicate that effective negative sampling should target nodes that are neither too close nor too far from the user, while also including some level of randomness in the selection. Compared to the baseline RNS approach, DD-NS-Long-Distance achieved a 2.3% improvement in Precision@10 (0.0811 to 0.083), 2.1% improvement in Recall@10 (0.1483 to 0.1515), and 2.5% improvement in NDCG@10 (0.1347 to 0.1382).

The DD-NS technique leverages the network’s diffusion process, which evolves with time. The parameter τ controls the diffusion time, essentially defining how far a random walk is allowed to explore the network. Lower τ values emphasize local similarities, resulting in low distances for directly connected nodes and high distances for distant ones. Conversely, higher τ values allow the exploration of longer paths, capturing more distant similarities by reducing distances for previously far nodes. We observed the effect of τ on the performance of DD-NS. For farthest, nearest, and quartile-based strategies, the performances are much lower than baselines and do not significantly change with τ . However, for long-distance strategy, we obtain better performance for any τ and scaling interval. Table 2 shows the performances of DD-NS-Long-Distance for different τ and scaling intervals. Accordingly, larger τ values require a wider scaling range for optimal performance, as they allow analysis over a broader network area. For example, the optimal τ is 0.5 for a scale of $[1, 5]$, and 4 for a scale of $[1, 100]$.

For exploring the answer to RQ2, we demonstrate the Recall@10, Precision@10, and NDCG@10 values of each epoch for the baseline RNS and proposed methods in Figure 3. Accordingly, the proposed network-oriented negative sampling approaches lead to improved model convergence compared to the baseline method, RNS. All proposed methods achieve faster and better convergence. On average, we observe a 30-35% improvement in convergence speed.

4.2. The Effect of Uniform Selection in Negative Sampling

In the traditional RNS, only one negative is selected per user, while our proposed approach selects the same number of negatives with the positives of each user. Thus, when we compare the results of our approaches with traditional RNS, there can be a misleading success/failure of our methods, which may be due to the number of sample that are used in model learning and not the strategy itself. We take into

Table 2. Results of DD-NS with Different Parameter Values in Last.fm Dataset

Sampling Approach	τ	Scaling Interval	Precision@10	Recall@10	NDCG@10
RNS	-	-	0.0811	0.1483	0.1347
PNS	-	-	0.0790	0.1435	0.1295
DD-NS-Long-Distance	0.5	[1,5]	0.0830	0.1515	0.1382
DD-NS-Long-Distance	1.0	[1,5]	0.0809	0.1474	0.1351
DD-NS-Long-Distance	2.0	[1,5]	0.0822	0.1492	0.1364
DD-NS-Long-Distance	4.0	[1,5]	0.0824	0.1502	0.1367
DD-NS-Long-Distance	0.5	[1,10]	0.0811	0.1477	0.1363
DD-NS-Long-Distance	1.0	[1,10]	0.0827	0.1505	0.1363
DD-NS-Long-Distance	2.0	[1,10]	0.0827	0.1494	0.1362
DD-NS-Long-Distance	4.0	[1,10]	0.0815	0.1481	0.1348
DD-NS-Long-Distance	0.5	[1,100]	0.0823	0.1489	0.1368
DD-NS-Long-Distance	1.0	[1,100]	0.0825	0.1483	0.1363
DD-NS-Long-Distance	2.0	[1,100]	0.0814	0.1467	0.1345
DD-NS-Long-Distance	4.0	[1,100]	0.0829	0.1500	0.1369

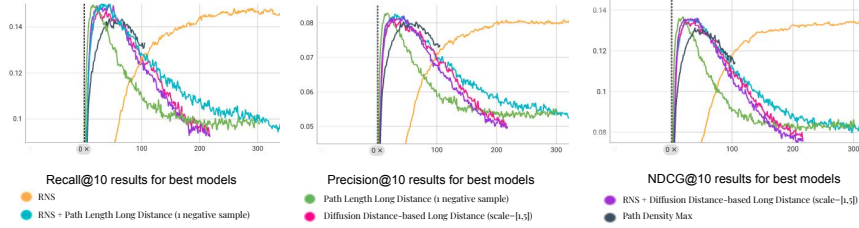


Fig. 3. The numbers of epochs that each proposed negative sampling and RNS has the best results with Recall@K, Precision@K, and NDCG@K

account this fact in our RQ3. To answer this, we investigated how our proposed negative sampling approaches perform when combined with uniformly selected negatives. In these experiments, we also overcome the factor of the unequal numbers of samples in RNS and our methods. We explored this on a 4-point scale, where:

- One extreme represents a completely uniform randomly selected negative item assignment for each positive item received by a user (Multi RNS).
- The other extreme represents negative item assignment based on our proposed network-oriented methods.

Within this scale, we created hybrid negative sampling methods for each of our proposed approaches. These methods combine uniform random and network-oriented elements: First, half of the negative items are selected completely randomly with uniform sampling. The remaining half are selected using our proposed network-oriented methods (Multi RNS + proposed approach). Second, we reduce the uniform

selection factor by only using one uniformly selected negative in combination with our proposed methods (RNS + proposed approach).

We conducted experiments for each of the hybrid approaches. However, due to space limitations, we only present the results for the path-length (PL) method in Table 3. Results for other methods followed a similar trend to the PL method. Accordingly, an increased number of uniformly selected negatives (Multi RNS) shows lower performance than the RNS baseline. Moreover, incorporating Multi RNS+PL approaches does not necessarily improve the results. In most cases, using only PL achieves better or similar performance compared to Multi RNS+PL and RNS+PL. This suggests that using path-length information alone might be sufficient to capture the negative sampling needs in this dataset. These results also indicate that the strategies themselves may cause an improvement in success, not the number of selected negatives. We can interpret this fact as meaning that for LightGCN, using the correct items for the learning phase can impact the result recommendations. Given the lengthy and costly learning curve of these methods, it may be worthwhile to take advantage of the topological properties of the bipartite network.

Table 3. Path-Length NS Approach Results in Last.fm Dataset

Sampling Approach	Precision@10	Recall@10	NDCG@10
RNS	0.0811	0.1483	0.1347
PNS	0.0790	0.1435	0.1295
Multi RNS	0.0796	0.1449	0.1314
Multi RNS+PL-Long-Distance	0.0814	0.1474	0.1344
Multi RNS+PL-Q1	0.0812	0.1479	0.1335
Multi RNS+PL-Q2	0.08	0.1438	0.1310
RNS+PL-Long-Distance	0.0827	0.1511	0.1361
RNS+PL-Q1	0.0805	0.1481	0.1351
RNS+PL-Q2	0.0773	0.1398	0.1277
PL-Long-Distance	0.0825	0.1496	0.1351
PL-Q1	0.0797	0.145	0.1324
PL-Q2	0.0771	0.1379	0.125

5. Conclusion

In this study, we address data sparsity and class imbalance, prevalent issues in RS. Our approaches leverage network topology to improve recommendation accuracy. To this end, we proposed ten novel negative sampling approaches based on network topological properties, including path length, diffusion distance, and path density. By focusing on the relative positions of items in the network, our methods identified

more relevant negative examples, improving the training process and recommendation accuracy.

Among the various proposed strategies, the long-distance prioritized weighted uniform random selection outperforms all baselines and other proposed techniques. Particularly the PL-Long-Distance approach increased the Precision@10 and NDCG@10 by 2% and 1.5%, respectively, compared to baseline uniform random sampling. In this view, our top-performing approach, DD-NS-Long-Distance, improved Precision@10, Recall@10, and NDCG@10 by up to 5%, 5.5%, and 6.7%, respectively, compared to the popularity-based negative sampling baseline. When compared to the uniform random negative sampling baseline, we observed enhancements of 2.3%, 2.1%, and 2.5% in Precision@10, Recall@10, and NDCG@10, respectively. These approaches have been much more successful than both the baselines and other approaches in which we select the closest or farthest items, with less randomness in them. Additionally, our strategies accelerated model convergence, reducing training time by up to 30-35%. This not only improves recommendation accuracy but also makes the approach computationally efficient for real-world RS.

Although the experiments in this study demonstrate the effectiveness of the proposed approaches, there are limitations. The proposed negative sampling strategies cause some processing costs since they examine the entire network to find the negative sample. Future work will investigate the use of local selection or mini-batch selection methodologies to alleviate these limitations and increase the scalability of comprehensive RS. Additionally, future research will focus on optimizing hyperparameters and evaluating these methodologies across diverse datasets to ensure generalizability and effectiveness in various domains, such as movie, hotel, and product recommendations.

Our work highlights the strong potential of incorporating complex network topology into GCN-based recommender systems to improve both the accuracy and efficiency of recommendation. By integrating topological features—such as path length and path density—into the negative sampling process of LightGCN, we enriched the learning data and demonstrated that structurally informed sampling can lead to more effective node embedding generation. While our current approach focuses on negative sample selection, the results suggest broader opportunities for integrating complex network concepts into the representation learning phase of modern recommendation architectures. These findings open promising avenues for future research on structurally aware learning strategies in deep learning-based recommender systems.

Acknowledgements

This work is supported by the bilateral project of the Scientific and Technological Research Council of Türkiye (TÜBİTAK), under grant number 122N701, with CampusFrance, within the scope of Hubert Curien Partnerships (PHC) project number 49032VB.

References

1. J. Bobadilla, F. Ortega, A. Hernando and A. Gutierrez, *Knowledge-Based Systems* **46**, 109 (2013).
2. S. Wu, F. Sun, W. Zhang, X. Xie and B. Cui, *ACM Comput. Surv.* **55** (dec 2022).
3. F. Ricci, L. Rokach and B. Shapira, *Recommender Systems Handbook* 10 2010, pp. 1–35.
4. M. E. J. Newman, *SIAM Review* **45**, 167 (2003).
5. M. Volkovs, G. W. Yu and T. Poutanen, Content-based neighbor models for cold start in recommender systems, in *Proceedings of the Recommender Systems Challenge 2017*, RecSys Challenge '17 (Association for Computing Machinery, New York, NY, USA, 2017).
6. X. Wang and Y. Wang, Improving content-based and hybrid music recommendation using deep learning, in *Proceedings of the 22nd ACM International Conference on Multimedia*, MM '14 (Association for Computing Machinery, New York, NY, USA, 2014).
7. Z. Yang, M. Ding, C. Zhou, H. Yang, J. Zhou and J. Tang, Understanding negative sampling in graph representation learning, in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '20 (Association for Computing Machinery, New York, NY, USA, 2020).
8. X. He, K. Deng, X. Wang, Y. Li, Y. Zhang and M. Wang, Lightgcn: Simplifying and powering graph convolution network for recommendation, in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '20 (Association for Computing Machinery, New York, NY, USA, 2020).
9. J. Wu, X. Wang, F. Feng, X. He *et al.*, Self-supervised graph learning for recommendation, in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '21 (Association for Computing Machinery, New York, NY, USA, 2021).
10. Z. Yang, M. Ding, T. Huang, Y. Cen *et al.*, *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024).
11. C. Wang, Z. Zhu, P. Meng and Y. Qiu, *Information Sciences* **606**, 853 (2022).
12. A. Grover and J. Leskovec, node2vec: Scalable feature learning for networks, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16 (Association for Computing Machinery, New York, NY, USA, 2016).
13. J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan and Q. Mei, Line: Large-scale information network embedding, in *Proceedings of the 24th International Conference on World Wide Web*, WWW '15 (International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 2015).
14. B. Perozzi, R. Al-Rfou and S. Skiena, Deepwalk: online learning of social representations, in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '14 (Association for Computing Machinery, New York, NY, USA, 2014).
15. T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado and J. Dean, Distributed representations of words and phrases and their compositionality, in *Advances in Neural Information Processing Systems*, eds. C. Burges, L. Bottou, M. Welling, Z. Ghahramani and K. Weinberger (Curran Associates, Inc., 2013).
16. T. Mikolov, K. Chen, G. Corrado and J. Dean, *Proceedings of Workshop at ICLR* **2013** (01 2013).
17. S. Rendle, C. Freudenthaler, Z. Gantner and L. Schmidt-Thieme, Bpr: Bayesian per-

18 WSPC

- sonalized ranking from implicit feedback, in *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, UAI '09 (AUAI Press, Arlington, Virginia, USA, 2009).
18. J. D. Rennie and N. Srebro, Loss functions for preference levels: Regression with discrete ordered labels, in *Proceedings of the IJCAI multidisciplinary workshop on advances in preference handling*, 2005.
 19. A. v. d. Oord, Y. Li and O. Vinyals, *arXiv preprint arXiv:1807.03748* (2018).
 20. L. Xiong, C. Xiong, Y. Li, K.-F. Tang *et al.*, Approximate nearest neighbor negative contrastive learning for dense text retrieval, in *International Conference on Learning Representations*, 2021.
 21. W. Zhang, T. Chen, J. Wang and Y. Yu, Optimizing top-n collaborative filtering via dynamic negative item sampling, in *Proceedings of the 36th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '13 (Association for Computing Machinery, New York, NY, USA, 2013).
 22. Z. Yang, M. Ding, X. Zou, J. Tang *et al.*, *IEEE Transactions on Knowledge and Data Engineering* **35**, 6264 (2023).
 23. C.-Y. Wu, R. Manmatha, A. J. Smola and P. Krähenbühl, Sampling matters in deep embedding learning, in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017.
 24. J. Rao, H. He and J. Lin, Noise-contrastive estimation for answer selection with deep neural networks, in *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, CIKM '16 (Association for Computing Machinery, New York, NY, USA, 2016).
 25. H. Zhang, M. Cisse, Y. N. Dauphin and D. Lopez-Paz, mixup: Beyond empirical risk minimization, in *International Conference on Learning Representations*, 2018.
 26. T. Huang, Y. Dong, M. Ding, Z. Yang *et al.*, Mixgcf: An improved training method for graph neural network-based recommender systems, in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, KDD '21 (Association for Computing Machinery, New York, NY, USA, 2021).
 27. J. Wang, L. Yu, W. Zhang, Y. Gong *et al.*, Irgan: A minimax game for unifying generative and discriminative information retrieval models, in *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '17 (Association for Computing Machinery, New York, NY, USA, 2017).
 28. D. H. Park and Y. Chang, Adversarial sampling and training for semi-supervised information retrieval, in *The World Wide Web Conference*, WWW '19 (Association for Computing Machinery, New York, NY, USA, 2019).
 29. C.-Y. Chuang, J. Robinson, Y.-C. Lin, A. Torralba and S. Jegelka, Debaised contrastive learning, in *Advances in Neural Information Processing Systems*, eds. H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan and H. Lin (Curran Associates, Inc., 2020).
 30. N. Masuda, M. A. Porter and R. Lambiotte, *Physics Reports* **716-717**, 1 (2017), Random walks and diffusion on networks.
 31. G. Bertagnolli and M. De Domenico, *Phys. Rev. E* **103**, p. 042301 (Apr 2021).
 32. B. Roberts and D. Kroese, *J. Graph Algorithms Appl.* **11**, 195 (01 2007).
 33. X. Glorot and Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, , Proceedings of Machine Learning Research Vol. 9 (PMLR, 13–15 May 2010).
 34. D. P. Kingma and J. Ba, *CoRR* **abs/1412.6980** (2014).